Thief & the Necklace: Game Design Document

Game Overview

Thief & the Necklace is a multiplayer social deduction card game for 4–8 players. Each player secretly receives a role card (e.g. King, Queen, Soldier), each with a point value. One random player is also dealt a special “Necklace” card, making them the Thief. Players engage in free-form discussion (no turns), trying to deduce who the Thief is. After a time, players vote on who they believe the Thief is. If the majority correctly identifies the Thief, those players earn points based on role cards; if not, the Thief earns the points. This mimics the classic social deduction format (players secretly have roles, then debate and vote to find hidden adversaries [1]).

- Platforms: Built in Unity for mobile (iOS/Android) and WebGL (browser). Note: Unity WebGL builds typically target desktop browsers; mobile browsers have limited support2. We will do separate builds for native mobile and for web.
- Multiplayer: Online rooms of 4–8 players. Lobby system for players to join before game start.
- Communication: Real-time text chat in MVP (e.g. Photon Chat). Future: voice chat (e.g. Photon Voice)
- UI/UX Style: A clean card-game interface. Key screens include Lobby, Game, Voting, and Results. Reveal/

Roles and Game Logic

Roles: Define a set of role cards (King, Queen, Soldier, etc.), each with a point value. For example, King=8, Queen=7, Soldier=5, etc. One player is secretly given the “Necklace” card (the Thief) instead of a normal role. The Thief’s objective is to avoid detection; others must find the Thief.

Game Phases:

1. Role Assignment: When the game starts, shuffle and deal one role card to each player. One random player also gets the Necklace (Thief) card hidden. Use a network-safe random method to assign roles.
2. Discussion (Free-form): Players can chat (text) freely to persuade others or cast suspicion. There are no turns — anyone may speak or type at any time. This mirrors the “day phase” in social deduction games where everyone discusses and debates1.
3. Voting: After a set discussion period or when players choose, enter the voting phase. Each player selects one person they suspect is the Thief. Tally votes across players.
4. Reveal and Scoring: Reveal the Thief’s identity.
- If the majority correctly voted for the Thief, those voting players earn points (for example, equal to their own role’s value or other game-defined formula). The Thief earns nothing.
- If the majority was wrong, the Thief earns points (e.g. the sum of all normal role values or a fixed reward). The wrongly accusing players earn nothing.
- Optionally, show each player’s role card and current score.
5. Next Round / End: Update cumulative scores. End the game after a fixed number of rounds or score target.

Example Logic: The exact scoring can be tuned. One approach: if Thief is caught, each correct guesser gets points equal to the value of their own role card; if Thief escapes, the Thief gets points equal to the

sum of all other players' role values. This ensures high-value roles matter. The key is clear win/lose conditions based on the vote. Players should be prevented from self-voting.

Development Roadmap

MVP (Basic Playable):

- Unity Setup: Create a new Unity project (choose a recent LTS version). Install required packages (e.g. UI Toolkit or TextMeshPro, networking SDK like Photon or Netcode, and a JSON library if needed). Set target platforms (mobile iOS/Android and WebGL).
- Core Logic: Implement the role assignment and voting logic locally first. Use simple C# scripts (e.g. `RoleManager`, `GameManager`) to shuffle roles, assign the Thief, manage discussion timer, collect votes, and calculate scoring. Keep UI simple (e.g. Unity UI Canvas with buttons and text).
- Networking: Integrate a basic multiplayer framework. For example, Photon PUN 2 offers cloud-hosted rooms and sync. Set up a lobby where players join a room, then instantiate the game scene when ready. Sync role distribution across clients and collect votes via network RPCs or raised events. (Photon Chat can be added for text messaging.)
- Text Chat: Implement a simple text chat window (input field + scroll list). Use Photon Chat or a custom solution to broadcast messages to all players in the room. Ensure chat messages are displayed in the UI.
- UI Screens: Build key scenes: Main Menu (with "Join/Create Room"), Lobby (show list of players, "Ready" button), Game (role reveal private UI, chat UI, timer), Voting (selection UI), Results (show thief and updated scores).
- Testing: Test the entire loop with 4–8 local instances. Use Unity's Play Mode or build copies to simulate players. Ensure roles are assigned correctly and synced, chat works, votes tally, scores update as expected [3]. and

Intermediate Features:

- Player UI: Add player avatars or icons, name tags, and role card graphics. • Robust Networking: Implement Photon callbacks (OnPlayerLeft, room list updates) or Netcode events for reliability. Handle disconnections gracefully (for MVP, maybe kick or wait).
- Matchmaking: Allow players to create or join random rooms (e.g. Photon's random matchmaking or list-of-rooms lobby).
- Scalability: Ensure room capacity limits (min 4, max 8). Show waiting message if too few players.
- Persistent Data: Add a simple local leaderboard or score tracking per session. Optionally integrate a backend (e.g. PlayFab or custom) for cross-session leaderboards.

Advanced Features:

- Voice Chat: Integrate a voice chat SDK (e.g. Photon Voice) so players can talk live. • Polish UI/UX: Add animations (card flips, vote highlight), sound effects, and polished art. Provide feedback (e.g. pinging suspected Thief on vote results).
- New Modes/Roles: Expand with additional role types or twist cards. Maybe a "Detective" role with special ability, etc.
- Cross-Platform Integration: Configure builds and input for different devices. • LiveOps: Deploy analytics (Unity Analytics) for tracking game events. Implement hotfix/update workflow.

Unity Architecture

Scenes:

- Main Menu Scene: Options to host or join a game. Settings. • Lobby Scene: Displays joined players, ready buttons, chat window. Handles player count and start game when ready.
- Game Scene: The main game table. Each client sees their own role card (face-up for them only) and others' face-down. Contains chat UI, discussion timer.
- Voting Scene (Overlay): When discussion ends, present a voting UI (e.g. selectable player portraits or buttons). This could be a separate overlay in the Game Scene.
- Reveal/Result Scene: Show all players' roles (who was Thief) and updated scores. Optionally allow rematch or return to lobby.

(Unity's docs note: "Think of each unique Scene file as a unique level… you will place your environments, obstacles, and decorations" in each Scene[4]. We use separate scenes for logically distinct screens.)

Prefabs:

Use Unity Prefabs to create reusable UI and game objects. For example: - Player Panel Prefab: Shows a player's name, avatar, status (ready/not ready), and later role. - Chat Message Prefab: An element for each chat message (author name + text). Instantiated under the chat scroll view. - Role Card Prefab: A card graphic with a script holding role data. Instantiate one per player. - Button Prefabs: Custom styled buttons for votes, ready, etc. - Manager Prefabs: If using services (e.g. Photon UnityNetworking object), include as persistent prefabs.

Using Prefabs ensures a consistent look and makes it easy to update UI elements across all scenes[5].

Scripts and Components:

- GameManager: A singleton (or persistent) script that controls the game flow: starts rounds, triggers discussion timer, moves to voting, calculates scores, etc.
- NetworkManager: Using Photon (or Netcode), handles connecting to server, creating/joining rooms, instantiating network objects.
- RoleAssigner: Logic to shuffle the role deck, randomly pick the Thief, and assign cards. On the network, this script ensures each client knows their own role.
- ChatController: Manages the chat UI and network messages. Listens for incoming chat events and creates message UI elements. Sends user input to others.
- VoteManager: Collects votes from each player and tallies them. Calls GameManager when voting is complete.
- ScoreManager: Updates and stores player scores. Could use a simple Dictionary<playerId, score>.
- UI Controllers: Separate scripts for each screen's UI (LobbyUI, GameUI, VoteUI, ResultUI) to handle user interactions (e.g. button clicks send network messages).

Keep scripts focused on one responsibility and use UnityEvents or observer patterns to decouple systems. For example, when a vote is cast, invoke an event that GameManager listens to.

Multiplayer System Design

Photon PUN 2 (Recommended MVP): Photon provides cloud-hosted multiplayer rooms with minimal setup. Players connect to Photon's cloud and join a room. Photon automatically syncs scene loads and can instantiate networked objects (e.g. player avatars). Use PhotonNetwork.JoinRandomRoom or room lists for matchmaking. Roles can be distributed on the MasterClient and sent via custom room properties or RPCs to each client. Voting results can be sent as RPCs to the master, which tallies and returns a result. For text chat, use Photon Chat (a separate Photon service) which allows quick integration of chat channels. Photon Chat is "the fast and easy way to add chat functionality" and a turnkey scalable solution[6][7]. For example, have all players subscribe to a room-specific chat channel so messages broadcast to only that room.

Unity Netcode for GameObjects: As an alternative, Unity's official Netcode (NGO) can be used. It requires more setup: typically you must host a server or use Unity Gaming Services (UGS) like Relay and Lobby. Netcode synchronizes state across clients (either client-hosted or server). You'd set one player as Host (or use a dedicated server build) and the others as Clients. Then use NetworkVariables or RPC calls to sync roles and votes. Unity's Netcode is designed for co-op multiplayer and can target multiple platforms[8]. Relay (UGS) lets you connect P2P without managing your own servers[9]. For a beginner, Photon is simpler to get started because of its managed cloud; Netcode/Relay is more integrated but has more steps (configuring UGS, etc).

Room/Match Design: Limit each room to 8 players. Enforce a minimum (e.g. 4) before starting. If a player disconnects early, decide whether to cancel or replace with AI/empty seat. Use Photon callbacks (e.g. OnPlayerLeftRoom) to handle drops. Security Note: You must trust clients to some extent or implement checks on the authoritative client. For example, the MasterClient or server should ultimately decide the Thief and scoring to prevent hacking. Use Photon's MasterClient or a Netcode server as authority.

Sample Script Structure (Pseudo-C#)

Below is a high-level outline of classes/scripts for key systems:

- RoleAssigner (MonoBehaviour)
- List<RoleCard> deck; int thiefIndex;
- void ShuffleAndDeal(): Randomize deck, pick one index as Thief (e.g. Random.Range(0, deck.Count)), then assign each deck[i] to player i via networked RPC or sync list.
- GameManager (MonoBehaviour) • States: enum Phase { WaitingForPlayers, Discussion, Voting, Reveal }
- Methods: StartGame(), BeginDiscussion(), BeginVoting(), RevealResult().
- Tracks phase transitions (e.g. with a coroutine timer for discussion length).
- Calls RoleAssigner.ShuffleAndDeal() at start.
- Receives votes from VoteManager, determines if the majority guess is correct, updates scores. • VoteManager (MonoBehaviour)

• Dictionary<Player, Player> votes; • UI presents player buttons; when a player clicks a vote, call PhotonNetwork.RaiseEvent(voterID, targetID).

• On receiving all votes, notify GameManager. • ChatController (MonoBehaviour) • UI:

InputField for new message, ScrollView for messages.
• void SendChatMessage(string text): Call Photon Chat API or raise a Photon event with

the

• On receiving chat messages, instantiate a Chat Message prefab with the content. •

ScoreManager (MonoBehaviour) • Dictionary<Player, int> scores;
• void AddScore(Player p, int amount): increments score.
• Called by GameManager after reveal. • NetworkManager/PhotonManager (MonoBehaviour) •

void ConnectAndJoin(): Connect to Photon, join lobby.
• void OnJoinedRoom(): Called when joined; instantiate local player UI, enable ready button.
• Relay network events (player joins, leaves, RPCs for game events).

These scripts should use Unity's networking callbacks (Photon callbacks or Netcode's NetworkBehaviours) to sync state. For example, in Photon you might use PhotonNetwork.Instantiate for player objects, and use PhotonView.RPC to send data. In Netcode, you'd use [ServerRpc] and [ClientRpc] methods or NetworkVariable<T> for shared state. Note: In all networked code, ensure logic for critical actions (like selecting the Thief) runs

only on the
server/host (e.g. MasterClient) to prevent cheating.

UI Mockups and Wireframes

Key Screens and Layout Guidance: - Main Menu: Options to Create Room or Join Room (with code). Settings icon. Keep it simple. - Lobby Screen: Shows a list of players (names/avatars) who have joined. Each entry shows Ready status. A "Ready" or "Start Game" button (Start enabled when min players ready). Include a chat panel on the side or bottom with scroll view of messages and input field. Use a simple list or grid layout for player slots. - Game Screen: Central area shows "table" with face-down role cards for each player. Each player sees their own card revealed (face-up). There should be a chat window (accessible at bottom or overlay) where discussion happens. A visible countdown timer and a button to call voting when time is up. Keep UI elements minimal so players focus on discussion. - Voting Overlay: When voting starts, display a dialog or overlay listing all player avatars/names as selectable buttons (except self). Possibly highlight last chatted players. Include a "Submit Vote" button. Once everyone votes or timer ends, close voting.

- Reveal/Results Screen: Show which player had the Necklace (Thief) with an icon. Display each player's role card briefly. Show points earned in this round and updated cumulative scores. Provide a "Next Round" or "Main Menu" button.

When designing, use Unity UI Canvas with careful layering. For example, separate canvases for chat, for main gameplay, and for popups. Use Unity's prefab system: create a prefab for the player entry in Lobby and for the vote button. (Unity docs on UI: a "Scene contains environments and menus"[4]. Prefabs help reuse UI elements[5].)

Chat Integration

For the MVP's real-time text chat, a turnkey solution like Photon Chat is ideal. Photon Chat is designed to "plug in within minutes" and scales with your user base[6]. You would do the following:

1. Sign Up for Photon Chat: Obtain an AppID from Photon's dashboard.
2. Add Chat SDK: Import Photon Chat SDK into Unity.
3. Connect to Chat: On entering the Lobby scene, connect to Photon Chat (e.g. `chatClient.Connect(...)`).
4. Channels: Have all players subscribe to a channel specific to the game room (e.g. `chatClient.Subscribe(new string[]{ roomName })`). This ensures messages are shared only within that room.
5. Send/Receive Messages: When a player types a message and hits send, publish it to the channel. Use the Photon Chat listener callbacks to receive new messages and display them in the UI.
6. UI: The ChatController script will instantiate message entries (from a prefab) under the scroll view when a new message arrives.

Later, for voice chat, Photon also offers Photon Voice. But initially, focus on stable text chat. Photon Chat is described as a "fast and easy way to add chat functionality" and a "turnkey solution to build a scalable chat" [6] [7], so integration is straightforward. Alternatively, Unity's own services or third-party (e.g. Discord SDK, Vivox) could be used, but Photon simplifies early development.

Optimization Tips for Mobile/Web

- Graphics Pipeline: Use the Universal Render Pipeline (URP) instead of the built-in pipeline. URP is optimized for performance on mobile and WebGL. Enable GPU Instancing only where supported.
- Asset Optimization: Compress textures (e.g. ASTC for Android, PVRTC for iOS) and use smaller resolutions. Simplify 3D models (fewer polygons) since on WebGL low-end hardware may struggle[10]. Use atlasing for UI sprites to reduce draw calls.
- Rendering: Minimize real-time lights; bake static lighting with lightmaps/light probes where possible[11]. Use occlusion culling so only visible objects render[12]. Use LOD groups for distant 3D models[13]. Turn off expensive post-processing effects in release builds[14].
- Code Efficiency: Use object pooling for frequently spawned objects (e.g. chat bubbles, cards) instead of Instantiate/Destroy to reduce garbage collection[15]. Avoid heavy string operations (use StringBuilder if needed)[16]. Keep scripts lean.
- UI Performance: Reduce the number of Canvas redraws. Group static and dynamic UI. Avoid full-screen canvases that change often.

• Build Settings: For WebGL, disable “Development Build” and enable Code Optimization (Link Time Optimization) to shrink binary size[17]. Use Brotli compression for the final build, as it produces smaller files than gzip[18].
• Addressables: Use Unity’s Addressable Asset System to load UI or art assets asynchronously at runtime rather than including everything in the initial load[19]. This reduces initial load time and memory. For example, load card graphics or background images on demand.
• Testing on Devices: Profile on real devices early. Use Unity’s Profiler or browser dev tools (for WebGL) to identify bottlenecks. Lower target frame rate if needed (e.g. 30 FPS on low-end).
• Quality Settings: Provide graphics quality levels (in Project Settings) and select a lower level for mobile or web if performance is poor.

These optimizations follow Unity’s recommendations for web and mobile: optimize assets and code size and speed[10], use modern rendering techniques (URP, SRP Batcher)[20], and compress builds for (Brotli, Addressables)[18] [19].

Testing Plan

1. Unit Testing (Logic): Write simple unit tests for core game logic (shuffling roles, vote tallying, score calculations) using a testing framework like NUnit. This is optional for a small project but helps catch logic bugs.
2. Play Mode Testing (Local): Regularly run multiple Unity instances. Unity’s Multiplayer Play Mode (a package) can simulate up to 4 players on one machine[3], which speeds up iteration. Alternatively, use the ParrelSync tool to clone the Editor for multiple clients. Verify that clients connect properly, roles sync correctly, chat messages appear everywhere, and votes are tallied. Test all edge cases (e.g. late join, disconnect, ties in voting).
3. Platform Testing: Build the game for each target: iOS, Android, and WebGL. Test performance and functionality on actual devices and browsers. WebGL in particular has unique constraints (memory, browser compatibility). Use browser developer tools to monitor network connections.
4. Network Conditions: Simulate poor network (high latency, packet loss) to see if the game still works (Photon has a network simulator in the Cloud Dashboard). For Netcode/Relay, use Unity’s Network Simulator in the editor. Ensure the game handles lag (e.g. by waiting for slow vote submissions).
5. Beta Testing: Release an alpha/beta to a small group (friends or testers). Collect feedback on UX (is the chat UI usable? are instructions clear?). Observe if any disconnections or UI bugs occur in real play.
6. Regression Testing: After each new feature (like voice chat), retest all existing features to ensure nothing broke.
7. Automation: (Advanced) For critical network code, consider automated tests by scripting multiple headless clients, but this can be complex for Unity. At minimum, ensure the build pipeline produces WebGL and mobile builds without errors via continuous integration (CI).

Testing should cover each stage: after the MVP is built, test all core gameplay thoroughly. After integrating chat, test chat under load (fast messages). After polish/UI changes, test on various screen sizes. Use the Unity Profiler and build reports to catch performance regressions. By following this structured roadmap and architecture, a solo beginner developer can incrementally build up Thief & the Necklace from a simple prototype to a full-featured social deduction game, keeping the project manageable. Remember to refer to Unity’s documentation for details on prefabs, and networking[4] [5], and to take advantage of community tutorials for Photon or Netcode integration as needed.

Sources: Unity documentation on Scenes and Prefabs[4][5]; social deduction game concepts (Mafia/ Werewolf)[1]; Photon Chat product page[6][7]; Unity Netcode overview[8][9]; Unity optimization guidelines[10][11][17][18][19]; Unity testing guidelines[3].

---

[1] Mafia (party game) - Wikipedia
https://en.wikipedia.org/wiki/Mafia_(party_game)

[2] WebGL Browser Compatibility - Unity - Manual
https://docs.unity3d.com/2020.1/Documentation/Manual/webgl-browsercompatibility.html

[3] Testing multiplayer games locally | Unity Multiplayer
https://docs-multiplayer.unity3d.com/netcode/current/tutorials/testing/testing_locally/

[4] Unity - Manual: Scenes
https://docs.unity3d.com/530/Documentation/Manual/CreatingScenes.html

[5] Unity - Manual: Prefabs
https://docs.unity3d.com/6000.1/Documentation/Manual/Prefabs.html

[6] [7] Worldwide Cross Platform Chat as a Service (SaaS, Cloud)
https://www.photonengine.com/chat

[8] [9] Networking & Netcode Software Solution | Unity
https://unity.com/products/netcode

[10] [11] [12] [13] [14] [15] [16] [20] How to profile and optimize a web build | Unity
https://unity.com/how-to/profile-optimize-web-build

[17] [18] [19] Unity - Manual: Optimize Web platform for mobile
https://docs.unity3d.com/6000.1/Documentation/Manual/web-optimization-mobile.html